รายงานโครงงาน

ToyPlay

เว็บแอปพลิเคชันร้านของเล่นออนไลน์
(E-Commerce Web Application)

พัฒนาด้วย Django Framework และ Django REST Framework

โครงงานนีพัฒนาระบบร้านค้าออนไลน์ครบวงจรสำหรับร้านของเล่น ตังแต่การสมัครสมาชิก เลือกซือสินค้าตามหมวดหมู่ ตะกร้า ชำระเงิน ใช้คูปองส่วนลด ให้คะแนนรีวิว บันทึก รายการโปรด ติดตามสถานะคำสังซือ ไปจนถึงระบบจัดการหลังบ้านสำหรับพนักงานและผู้ดูแล พร้อม REST API สำหรับต่อยอด และ Deploy ขึนออนไลน์บน Render ได้จริง

รายการ	รายละเอียด
ชือโครงงาน	ToyPlay - เว็บแอปพลิเคชันร้านของเล่น (Django Web Application)
ประเภทโครงงาน	เว็บแอปพลิเคชัน / E-Commerce
เฟรมเวิร์ก	Django 6.0.6 + Django REST Framework
ฐานข้อมูล	SQLite (พัฒนา) / PostgreSQL (Production บน Render)
ผู้จัดทำ	นาย/นางสาว _____
อาจารย์ทีปรึกษา	_____
สถาบัน	มหาวิทยาลัยราชภัฏกำแพงเพชร (KPRU)
วันทีจัดทำ	14 ตุลาคม 2569

บทคัดย่อ

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนาระบบร้านของเล่นบนเว็บแอปพลิเคชันด้วยเฟรมเวิร์ก Django โดยระบบถูกออกแบบให้เป็นร้านค้าออนไลน์ที่ใช้งานได้ครบวงจร ประกอบด้วยฟีเจอร์ การสมัครสมาชิกและเข้าสู่ระบบ การแสดงสินค้าตามหมวดหมู่ การค้นหา กรอง และเรียงลำดับ สินค้า การแบ่งหน้า (Pagination) ตะกร้าสินค้า การชำระเงินพร้อมคูปองส่วนลดและการตัด สต็อกอัตโนมัติ การให้คะแนนและรีวิวสินค้า รายการโปรด (Wishlist) และการติดตามสถานะ คำสั่งซื้อแบบ timeline
ระบบรองรับผู้ใช้ 3 บทบาท ได้แก่ สมาชิก (member) พนักงาน (staff) และผู้ดูแลระบบ (admin) โดยมีการกำหนดสิทธิ์การเข้าถึงตามบทบาทจริงผ่าน decorators และ Django Admin นอกจากนี้ยังมีระบบหลังบ้าน (Dashboard) สำหรับดูสถิติยอดขาย จัดการสินค้า หมวดหมู่ สมาชิก และคำสั่งซื้อ พร้อมให้บริการ REST API ผ่าน Django REST Framework โดยใช้ Token Authentication สำหรับการเชื่อมต่อจากแอปพลิเคชันอื่น
ระบบได้รับการทดสอบอัตโนมัติด้วย Django TestCase ทั้งหมด 21 รายการ ครอบคลุมการสมัคร และเข้าสู่ระบบ การควบคุมสิทธิ์ การเพิ่ม/แก้ไข/ลบสินค้า การค้นหาและแบ่งหน้า หมวดหมู่ รีวิว รายการโปรด คูปอง การตัดสต็อก และการสร้างผู้ดูแลระบบจาก Environment Variables ผลการทดสอบผ่านทั้งหมด
และระบบพร้อมนำขึ้นออนไลน์บนแพลตฟอร์ม Render ผ่าน HTTPS

คำสำคัญ: Django, Django REST Framework, ร้านค้าออนไลน์, E-Commerce, ระบบหลังบ้าน, REST API

1. บทนำ

1.1 ชื่อโครงงาน

ToyPlay - ระบบเว็บแอปพลิเคชันร้านของเล่น พัฒนาด้วย Django Framework

1.2 วัตถุประสงค์

- เพื่อพัฒนาระบบเว็บแอปพลิเคชันร้านค้าออนไลน์ด้วย Django Framework ตามเกณฑ์การประเมินโครงงาน
- เพื่อประยุกต์ใช้ Django Model, Template, View และ URL routing ในการพัฒนาระบบจริง
- เพื่อฝึกออกแบบและกำหนดสิทธิ์ผู้ใช้อย่างน้อย 2 บทบาท (สมาชิก / พนักงาน / ผู้ดูแลระบบ)
- เพื่อฝึกการนำระบบขึ้น Hosting และให้เข้าใช้งานได้ผ่านอินเทอร์เน็ต
- เพื่อฝึกการทดสอบระบบอัตโนมัติ (Automated Testing) และการจัดทำรายงานโครงงาน

1.3 ขอบเขตของระบบ

- ระบบรองรับการสมัครสมาชิก เข้าสู่ระบบ และออกจากระบบ
- ระบบมีผู้ใช้ 3 บทบาท: สมาชิก (member), พนักงาน (staff) และผู้ดูแลระบบ (admin)
- สมาชิกสามารถดูสินค้าตามหมวดหมู่ ค้นหา กรอง/เรียง ใส่ตะกร้า ใช้คูปอง และสั่งซื้อได้
- สมาชิกสามารถรีวิวให้คะแนน บันทึกรายการโปรด และติดตามสถานะคำสั่งซื้อของตนเองได้
- พนักงาน/ผู้ดูแลสามารถเพิ่ม แก้ไข ลบสินค้า จัดการหมวดหมู่ และจัดการคำสั่งซื้อได้
- ผู้ดูแลระบบสามารถจัดการบัญชีสมาชิกและพนักงาน รวมถึงกำหนดสิทธิ์การใช้งานได้
- ระบบมี REST API (Token Authentication) สำหรับเข้าถึงข้อมูลสินค้า ตะกร้า และคำสั่งซื้อ
- ระบบ Deploy บน Render และเข้าใช้งานได้ผ่าน HTTPS

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- ผู้ใช้ทั่วไปสามารถเลือกซื้อของเล่นผ่านเว็บได้อย่างสะดวก
- พนักงานและผู้ดูแลร้านสามารถจัดการสต็อกสินค้าและคำสั่งซื้อได้อย่างเป็นระบบ
- ร้านค้าได้ข้อมูลสถิติยอดขาย สินค้าขายดี และสินค้าใกล้หมด เพื่อช่วยในการตัดสินใจ
- ผู้พัฒนาได้ฝึกทักษะ Full Stack ทั้งส่วนหน้าบ้าน (Frontend) หลังบ้าน (Backend) ฐานข้อมูล และการ Deploy

2. การวิเคราะห์และออกแบบระบบ

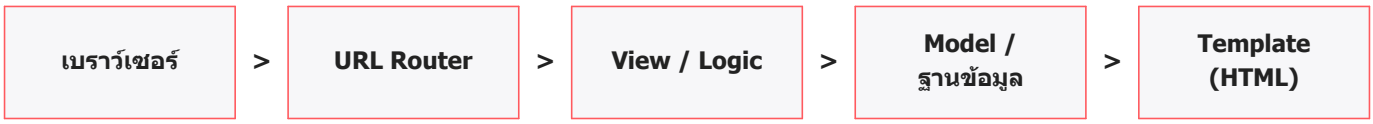
2.1 ภาพรวมและกรณีศึกษา

กรณีศึกษาคือร้านของเล่น ToyPlay ซึ่งมีสินค้าหลายหมวดหมู่ ลูกค้าสามารถดูรายละเอียด สินค้า ค้นหา เปรียบเทียบ และสั่งซื้อผ่านเว็บไซต์ได้ ฝ่ายร้านค้าสามารถจัดการข้อมูลสินค้า หมวดหมู่ สมาชิก และคำสั่งซื้อ ตลอดจนดูสถิติยอดขายผ่านหน้าหลังบ้าน

2.2 ผู้ใช้และบทบาท (User Roles)

บทบาท	รหัส	สิทธิ์การใช้งานหลัก
ลูกค้า / สมาชิก	member	ดูสินค้า, ค้นหา, ตะกร้า, ใช้คูปอง, checkout, รีวิว, รายการโปรด, ดูประวัติคำสั่งซื้อของตนเอง
พนักงาน	staff	สิทธิ์สมาชิก + เข้าหน้าลังบ้าน, จัดการสินค้า/หมวดหมู่ และคำสั่งซื้อ (is_staff = True)
ผู้ดูแลระบบ	admin	สิทธิ์สูงสุด รวมถึงจัดการบัญชีสมาชิก/พนักงานและกำหนดสิทธิ์ (is_superuser = True)

2.3 สถาปัตยกรรมโดยรวมของระบบ



ระบบใช้รูปแบบสถาปัตยกรรม MTV (Model-Template-View) ของ Django โดยเบราว์เซอร์ส่ง คำขอเข้าสู่ URL Router ซึ่งจะเรียก View ทีทำหน้าทีประมวลผล ติดต่อฐานข้อมูลผ่าน Model และแสดงผลกลับเป็นหน้า HTML ผ่าน Template ส่วน REST API จะตอบกลับเป็นข้อมูล JSON

2.4 โครงสร้างฐานข้อมูล (Data Models)

โมเดล	ความสัมพันธ์	คำอธิบาย
Category	-	หมวดหมู่สินค้า (ชื่อไม่ซ้ำ)
Product	FK -> Category	สินค้า: ชื่อ, หมวดหมู่, รายละเอียด, ราคา, สต็อก, สถานะ, รูปภาพ
Review	FK -> Product / User	รีวิวและคะแนน 1-5 ดาว (1 ผู้ใช้รีวิว 1 สินค้าได้ครั้งเดียว)
Wishlist	FK -> User / Product	สินค้ารายการโปรดของผู้ใช้
Coupon	-	คูปองส่วนลดแบบเปอร์เซ็นต์, ยอดขั้นต่ำ, จำกัดจำนวนครั้ง, วันหมดอายุ
Cart	OneToOne -> User	ตะกร้าสินค้าของสมาชิก
CartItem	FK -> Cart / Product	รายการสินค้าในตะกร้า และจำนวน
Order	FK -> User / Coupon	คำสั่งซื้อ: สถานะ, ส่วนลด, ยอดรวม
OrderItem	FK -> Order / Product	รายการสินค้าในคำสั่งซื้อ (เก็บชื่อ/ราคา ณ ตอนสั่งซื้อ)
UserProfile	OneToOne -> User	โปรไฟล์สมาชิก: รูปโปรไฟล์, เบอร์โทร, ที่อยู่
EmployeeProfile	OneToOne -> User	ข้อมูลพนักงาน: ตำแหน่ง, เบอร์โทร

ระบบใช้ความสัมพันธ์ One-to-One (โปรไฟล์/ตะกร้ากับผู้ใช้) One-to-Many (ผู้ใช้กับคำสั่งซื้อ, คำสั่งซื้อกับรายการสินค้า) และ Many-to-One (สินค้ากับหมวดหมู่) โดยเมื่อผู้ใช้ใหม่ถูกสร้าง ระบบจะสร้าง UserProfile และ Cart ให้อัตโนมัติผ่าน signal (post_save)

2.5 สถานะของคำสั่งซื้อ (Order Status)

สถานะ	ความหมาย
pending	รอดำเนินการ (ค่าเริ่มต้นหลังสั่งซื้อ)
confirmed	ยืนยันคำสั่งซื้อแล้ว
shipping	กำลังจัดส่ง
completed	สำเร็จ (ได้รับสินค้าแล้ว)
cancelled	ยกเลิก

ลูกค้าสามารถติดตามสถานะแบบ timeline ได้ตามลำดับ: รอดำเนินการ > ยืนยันคำสั่งซื้อ > กำลังจัดส่ง > สำเร็จ (หรือยกเลิก)

3. การพัฒนาระบบ

3.1 เทคโนโลยีที่ใช้

ส่วนของระบบ	เทคโนโลยี / เครื่องมือ
ภาษาและเฟรมเวิร์ก	Python 3, Django 6.0.6
REST API	Django REST Framework + Token Authentication
ฐานข้อมูล	SQLite (พัฒนา) / PostgreSQL (Production) ผ่าน dj-database-url
ส่วนติดต่อผู้ใช้	HTML, Bootstrap 5.3.3, CSS (static/css/style.css)
จัดการ Static/Media	WhiteNoise + Django staticfiles, Pillow (อัปโหลดรูปภาพ)
เว็บเซิร์ฟเวอร์	Gunicorn (WSGI)
การ Deploy	Render (เชื่อมกับ GitHub อัตโนมัติ) + render.yaml
การทดสอบ	Django TestCase (21 เทสต์ ผ่านทั้งหมด)

3.2 โครงสร้างโปรเจกต์

โฟลเดอร์ / ไฟล์	หน้าที่
myproject/	ตั้งค่าโปรเจกต์: settings.py, urls.py, wsgi.py, asgi.py
myapp/models.py	โมเดลฐานข้อมูลทั้งหมด (Product, Order, Coupon ฯลฯ)
myapp/views.py	ตรรกะการทำงานฝั่งเว็บ (Storefront + Dashboard + Cart + Checkout)
myapp/forms.py	ฟอร์มและ validation (Register, Product, StaffUser, Profile, Category, Review)
myapp/api_views.py	REST API views (products, auth, cart, orders)
myapp/serializers.py	ตัวแปลงข้อมูล Model เป็น JSON สำหรับ API
myapp/urls.py + api_urls.py	เส้นทาง URL ของเว็บและ API
myapp/templates/	เทมเพลต HTML (home, product, cart, dashboard, profile ฯลฯ)
myapp/migrations/	ประวัติการเปลี่ยนแปลงฐานข้อมูล + seed ข้อมูลเริ่มต้น
myapp/tests.py	ชุดทดสอบอัตโนมัติ 21 รายการ
media/	ไฟล์รูปภาพที่ผู้ใช้อัปโหลด (สินค้า, avatar)

3.3 ระบบยืนยันตัวตนและสิทธิ์

ระบบใช้ Django Authentication สำหรับการสมัครสมาชิก (RegisterForm), เข้าสู่ระบบ และ ออกจากระบบ โดยใช้ decorator @login_required และ @user_passes_test เพื่อจำกัดการเข้าถึง หน้าหลังบ้านเฉพาะ staff/admin ซึ่งตรวจสอบผ่านฟังก์ชัน is_staff_user และ is_admin_user

3.4 การจัดการข้อมูล (CRUD) และฟอร์ม

- สินค้า: เพิ่ม/แก้ไข/ลบ ผ่าน ProductForm (ModelForm) ในหน้า Dashboard
- หมวดหมู่: เพิ่มหมวดหมู่ใหม่ผ่าน CategoryForm
- สมาชิก/พนักงาน: เพิ่ม/แก้ไข/ลบ และกำหนดสิทธิ์ผ่าน StaffUserForm (เฉพาะ admin)
- โปรไฟล์: แก้ไขรูปโปรไฟล์ เบอร์โทร ที่อยู่ และอีเมลผ่าน ProfileForm
- ทุกฟอร์มมี validation เช่น ราคา/สต็อกต้องไม่ติดลบ, รหัสผ่านยาวอย่างน้อย 6 ตัวอักษร, ชื่อผู้ใช้ซ้ำไม่ได้

3.5 การค้นหา กรอง เรียง และแบ่งหน้า

- ค้นหาสินค้าจากชื่อและรายละเอียดด้วย Q objects (พารามิเตอร์ q)
- กรองตามหมวดหมู่ (category) และเรียงตามราคา/ความใหม่ (sort)
- ค้นหาสมาชิก/สินค้าใน Dashboard ด้วยพารามิเตอร์ pq / uq
- ใช้ Django Paginator แบ่งหน้า (ITEMS_PER_PAGE = 6 สำหรับหน้าเว็บ, API PAGE_SIZE = 10)

3.6 ตะกร้า การชำระเงิน และคูปอง

- เพิ่มสินค้าลงตะกร้า ปรับจำนวน และลบรายการ โดยตรวจสอบสต็อกไม่ให้สั่งเกินจำนวนที่มี
- ยืนยันการสั่งซื้อด้วย transaction.atomic เพื่อป้องกันข้อมูลไม่สมบูรณ์
- รองรับคูปองส่วนลดแบบเปอร์เซ็นต์ พร้อมเงื่อนไขยอดขั้นต่ำ วันหมดอายุ และจำนวนครั้งที่ใช้ได้
- ตัดสต็อกสินค้าอัตโนมัติเมื่อสั่งซื้อสำเร็จ และบันทึกยอดรวม/ส่วนลดลงในคำสั่งซื้อ
- เก็บชื่อและราคาสินค้า ณ ตอนสั่งซื้อไว้ใน OrderItem เพื่อรักษาประวัติแม้ราคาสินค้าเปลี่ยนภายหลัง

3.7 รีวิวและรายการโปรด

สมาชิกที่เข้าสู่ระบบสามารถให้คะแนนรีวิวสินค้า 1-5 ดาว พร้อมความคิดเห็น โดย 1 ผู้ใช้ สามารถรีวิวสินค้าเดิมได้เพียงครั้งเดียวและแก้ไขได้ (update_or_create) ส่วนรายการโปรด รองรับการกดหัวใจ เพิ่ม/ลบสินค้าที่ชื่นชอบ และแสดงในหน้าของผู้ใช้

3.8 REST API (ส่วนเสริม)

ระบบให้บริการ REST API ผ่าน Django REST Framework โดยใช้ Token Authentication สำหรับไคลเอนต์ภายนอก และรองรับ Session Authentication สำหรับผู้ใช้ทีล็อกอินในเว็บ การแก้ไขข้อมูลสินค้าจำกัดเฉพาะ staff/admin

Endpoint	Method	คำอธิบาย	สิทธิ์
/api/products/	GET, POST	ดู/เพิ่มรายการสินค้า	อ่าน: ทุกคน, เขียน: staff
/api/products/<id>/	GET, PUT, PATCH, DELETE	ดู/แก้ไข/ลบสินค้า	เขียน: staff
/api/auth/register/	POST	สมัครสมาชิกผ่าน API (คืนค่า token)	ทุกคน
/api/auth/login/	POST	ขอ token สำหรับเข้าสู่ระบบ	ทุกคน
/api/auth/me/	GET	ข้อมูลผู้ใช้ปัจจุบัน	ล็อกอิน
/api/cart/	GET, POST	ดู/เพิ่มสินค้าในตะกร้า	ล็อกอิน
/api/cart/items/<id>/	DELETE	ลบรายการในตะกร้า	ล็อกอิน
/api/orders/	GET, POST	ดูคำสั่งซื้อ / สร้างคำสั่งซื้อจากตะกร้า	ล็อกอิน
/api/orders/<id>/	GET	ดูรายละเอียดคำสั่งซื้อ	ล็อกอิน (เจ้าของ)

3.9 Dashboard และสถิติหลังบ้าน

- ยอดขายรวม (ไม่รวมคำสั่งซื้อทียกเลิก), จำนวนคำสั่งซื้อทั้งหมด และคำสั่งซื้อรอดำเนินการ
- จำนวนสมาชิกทั้งหมด จำนวนสินค้าทั้งหมด และจำนวนสินค้าใกล้หมด (สต็อก <= 5)
- สินค้าขายดี 5 อันดับ (Top Products) จากยอดขายสะสม และรายการคำสั่งซื้อล่าสุด
- หน้าจัดการคำสั่งซื้อ: ค้นหา/กรองตามสถานะ และอัปเดตสถานะคำสั่งซื้อ

3.10 Django Admin

เปิดใช้งาน Django Admin ที /admin/ สำหรับจัดการข้อมูลทุกตาราง (สินค้า หมวดหมู่ รีวิว รายการโปรด คูปอง คำสั่งซื้อ และผู้ใช้) พร้อมค้นหาและตัวกรอง โดยผู้ดูแลระบบจะถูกสร้าง หรืออัปเดตอัตโนมัติจาก Environment Variables (DJANGO_SUPERUSER_USERNAME / DJANGO_SUPERUSER_PASSWORD) ทุกครั้งทีรัน migrate

4. ผลการพัฒนาและภาพรวมหน้าจอ

ระบบสามารถรันได้ทั้งในเครื่อง (Local) และบน Render โดยเข้าใช้งานได้ผ่าน HTTPS
ต่อไปนีเป็นภาพรวมของหน้าจอสำคัญในระบบ (สามารถแทรกภาพหน้าจอจริงได้ในกรอบด้านล่าง)

หน้าจอ	รายละเอียด
หน้าแรก (Home)	hero banner, หมวดหมู่สินค้า, สินค้าแนะนำพร้อมคะแนนรีวิว
หน้าสินค้า (Products)	ค้นหา กรองตามหมวดหมู่ เรียงลำดับ และแบ่งหน้า
รายละเอียดสินค้า	รูปภาพ ราคา สต็อก เรตติง รีวิว สินค้าที่เกี่ยวข้อง และปุ่มรายการโปรด
ตะกร้า / ชำระเงิน	ปรับจำนวน ใช้คูปอง และยืนยันคำสั่งซื้อ
โปรไฟล์สมาชิก	อัปโหลดรูปโปรไฟล์ แก้ไขที่อยู่ และดูประวัติคำสั่งซื้อ + ติดตามสถานะ
Dashboard หลังบ้าน	สถิติยอดขาย สินค้าขายดี สินค้าใกล้หมด และจัดการสินค้า/สมาชิก
จัดการคำสั่งซื้อ	ค้นหา กรองตามสถานะ และอัปเดตสถานะคำสั่งซื้อ
REST API	ทดสอบ request/response และสถานะ HTTP ผ่าน Postman

ตัวอย่างภาพหน้าจอ (แทรกภาพจริง)

[ภาพหน้าจอ 1: หน้าหลัก ToyPlay]

[ภาพหน้าจอ 2: หน้าสินค้าและ Dashboard จัดการสินค้า]

[ภาพหน้าจอ 3: REST API /api/products/]

5. การทดสอบและสรุปผล

5.1 ชุดการทดสอบอัตโนมัติ

ระบบใช้ Django TestCase ในการทดสอบอัตโนมัติทั้งหมด 21 รายการ โดยรันคำสั่ง python manage.py test และผลการทดสอบผ่านทั้งหมด

กลุ่มการทดสอบ	จำนวน	ขอบเขตที่ทดสอบ	ผล
AuthRoleTests	2	สมัคร/เข้าสู่ระบบ และควบคุมสิทธิ์เข้า Dashboard	ผ่าน
ProductCrudTests	3	เพิ่ม/แก้ไข/ลบสินค้า	ผ่าน
SearchPaginationTests	3	ค้นหา แบ่งหน้า และแสดงสถิติ	ผ่าน
CategoryFilterTests	3	กรอง/เรียงตามหมวดหมู่ และเพิ่มหมวดหมู่	ผ่าน
ReviewWishlistTests	4	รีวิว (เพิ่ม/แก้ไข) และรายการโปรด	ผ่าน
CheckoutCouponStockTests	3	ตัดสต็อก ใช้คูปอง และกันสั่งเกินสต็อก	ผ่าน
SuperuserSyncTests	3	สร้าง/อัปเดตแอดมินจาก Environment Variables	ผ่าน
รวม	21	ครอบคลุมทุกฟีเจอร์หลักของระบบ	ผ่านทั้งหมด

5.2 ผลการทดสอบเพิ่มเติม

- python manage.py check: ไม่พบข้อผิดพลาด
- ทดสอบสิทธิ์: สมาชิกทั่วไปไม่สามารถเข้าถึง Dashboard ได้ (ถูก redirect)
- ทดสอบ API: สมาชิกทั่วไปที่พยายามเพิ่มสินค้าผ่าน API จะถูกปฏิเสธ (ไม่มีสิทธิ์)
- ทดสอบบน Render: เข้าใช้งานได้จริงผ่าน HTTPS

5.3 ข้อจำกัด

- ข้อมูลบนแพลนฟรีของ Render อาจไม่คงอยู่เมื่อ redeploy หากไม่ใช้ฐานข้อมูลภายนอกถาวร
- ไฟล์ media (รูปภาพ) บนแพลนฟรีเป็นแบบชั่วคราว (ephemeral) ไม่ถาวร
- ยังไม่มีระบบชำระเงินออนไลน์จริง (Payment Gateway) และยังไม่รองรับหลายภาษา

5.4 แนวทางพัฒนาต่อ

- เชื่อมต่อ PostgreSQL แบบถาวรและใช้ Cloud Storage สำหรับไฟล์ media
- เพิ่มระบบชำระเงินออนไลน์ (เช่น PromptPay / บัตรเครดิต)
- เพิ่มระบบแจ้งเตือนอีเมล และรายงานสถิติเชิงลึก
- เพิ่มเอกสาร API อัตโนมัติ (Swagger/OpenAPI) และระบบ CI/CD

5.5 สรุป

ระบบ ToyPlay ตอบโจทย์เกณฑ์ขั้นต่ำของโครงงานครบถ้วน ทั้งการจัดการข้อมูล CRUD การค้นหา การแบ่งหน้า การอัปโหลดรูปภาพ ระบบสมาชิกและสิทธิ์ผู้ใช้หลายบทบาท และการ Deploy บน Hosting
อีกทั้งยังพัฒนาต่อยอดด้วยระบบหลังบ้านพร้อมสถิติ คูปองส่วนลด การติดตามสถานะ คำสั่งซื้อ REST API และชุดทดสอบอัตโนมัติ 21 รายการ เพื่อเพิ่มความน่าเชื่อถือและรองรับคะแนนพิเศษ

6. ภาคผนวก

A. URL ของระบบ

- Production URL: https://toyplay-django.onrender.com/
- Health Check: https://toyplay-django.onrender.com/health/
- GitHub Repository: https://github.com/rungrueng9scream-arch/toyplay-django

B. บัญชีทดสอบ

บทบาท	ชื่อผู้ใช้ (Username)	รหัสผ่าน (Password)
ผู้ดูแลระบบ (admin)	boss	1
พนักงาน (staff)	staff / admin (สร้างผ่าน Dashboard)	กำหนดเอง
สมาชิก (member)	สมัครใหม่ได้ที่หน้า /register/	อย่างน้อย 6 ตัวอักษร

คูปองทดลองใช้: WELCOME10 (ลด 10%) และ TOY20 (ลด 20% ขั้นต่ำ 1,000 บาท)

C. การรันในเครื่อง (Local)

1. python -m venv .venv แล้ว activate
2. pip install -r requirements.txt
3. python manage.py migrate
4. python manage.py runserver แล้วเปิด http://127.0.0.1:8000
5. python manage.py test เพื่อรันชุดทดสอบอัตโนมัติ

D. Environment Variables (Production)

ตัวแปร	ค่า / คำอธิบาย
SECRET_KEY	คีย์ลับของ Django (สุ่มเป็นสตริงยาว)
DEBUG	False (ปิดโหมด debug บน Production)
ALLOWED_HOSTS	โดเมนของแอป เช่น your-app.onrender.com
DATABASE_URL	Internal Database URL ของ PostgreSQL จาก Render
DATABASE_SSL_REQUIRE	True เมื่อใช้ PostgreSQL บน Render
DJANGO_SUPERUSER_USERNAME	ชื่อผู้ใช้แอดมิน (สร้าง/อัปเดตอัตโนมัติ)
DJANGO_SUPERUSER_PASSWORD	รหัสผ่านแอดมิน
DJANGO_SUPERUSER_EMAIL	อีเมลแอดมิน (ไม่บังคับ)

Build Command: pip install -r requirements.txt && python manage.py collectstatic --noinput && python manage.py migrate
Start Command: gunicorn myproject.wsgi:application

จัดทำโดย นักศึกษา KPRU - 14 ตุลาคม 2569